NTNU
Norwegian University of Science and Technology (NTNU)
Faculty of Information Technology and Electrical Engineering
Department of Engineering Cybernetics

Master's Thesis

Candidate: **Bukueva Elena**

Course: **TTK4900 Master's Thesis**

Title (English): **Compiler-Aware Model Optimization for Edge AI Accelerators**

Title (Norwegian): **Kompilatorbevisst modelloptimalisering for Edge AI-akseleratorer**

Thesis description:

The tasks will be:

1. Develop a framework for evaluating deep learning models across deployment configurations in terms of accuracy, latency, and resource usage.
2. Explore model optimization techniques to improve computational efficiency while maintaining acceptable performance.
3. Analyze optimized models on target hardware, identify bottlenecks, and investigate ways to improve execution efficiency.

Start date: 15. January, 2026

Due date: 08. June, 2026

Thesis performed at: Department of Engineering Cybernetics

Supervisor: Professor Geir Mathisen, Dept. of Eng. Cybernetics

Abstract

Sammendrag

Contents

Abbreviations

1 Introduction

1.1 Motivation

1.2 Limitations

1.3 Contributions

Within these limitations, the main contributions of this thesis are:

- :
- :
- :
- :
- :

1.4 Thesis Structure

The remainder of this report is organised as follows:

- **Chapter 2**
- **Chapter 3**
- **Chapter 4**
- **Chapter 5**
- **Chapter 6**
- **Chapter 7**
- **Chapter 8**
- **Chapter 9**

- **Chapter 10**
- **Appendix A**
- **Appendix B**

2 Literature Study

2.1 Introduction to Object Detection with Deep Learning Models

Artificial Intelligence (AI) is a broad term covering a set of algorithms and operations on matrices and vectors that enable computers to perform a big variety of tasks in language understanding and translation, object recognition, robotic performance etc. We will consider only models performing object detection tasks, mostly having CNN [1] and Transformer [2] architectures.

2.2 Introduction to Deep Learning Model Optimization

2.3 Yolo8 Model Architecture

Ultralytics YOLOv8 model follows the classic Backbone -¿ Neck -¿ Head layout with some improvements from other YOLO architectures.

The key role of the Backbone is feature extraction. It converts the image into hierarchical feature maps (from low to high level).

Its key components are:

- Conv + BN + SiLU
- C2f blocks (main innovation)
- SPPF

The core idea of the C2f (Cross-Stage Partial with full feature fusion)

A bottleneck is a small stack of layers that:

1. Compresses channels(make tensor narrower)
2. Does some work (spatial mixing)

3. Expands channels back

It helps to drop redundant features, while preserving important.

SPPF = Spatial Pyramid Pooling – Fast

It's a context aggregation block used in YOLO (v5 → v8)

2.4 Performance Challenges in Machine Learning Deployment

Metrics:

- **Latency**: The time needed to process one inference request (etc. picture, text query);
- **Throughput**: The number of inference requests processed per unit of time (e.g., inferences per second).
- **Power Consumption**: The energy required to perform inference
- **Memory Requirements**: The amount of RAM and storage required for the model weights and intermediate activations

With the growing interest in on-device deployment, a clear gap has emerged between the training and inference stages of machine learning models. While large models are typically trained in data centers using powerful GPUs, inference is performed across a wide variety of deployment environments. Depending on the application, inference may be subject to strict constraints on memory, latency, and power consumption, and must run on heterogeneous hardware platforms such as CPUs, GPUs, TPUs, NPUs, or FPGAs. Each of these hardware architectures differs in structure, instruction sets, memory hierarchies, and supported data types. As a result, model performance depends heavily on how well it is adapted to the target hardware.

Machine learning models are usually developed in high-level Python frameworks such as TensorFlow or PyTorch. These frameworks often use dynamic execution (for example, PyTorch's eager mode), where:

- Operations are executed immediately, line by line
- Control flow (loops, conditionals) is resolved at runtime

- Tensors, layers, and operations are created and destroyed on the fly

That same flexibility comes at a cost:

- Python interpreter overhead Every operation goes through the Python interpreter, which is relatively slow.
- Interpreter locks (e.g., the GIL) Python's Global Interpreter Lock can limit parallel execution.
- Object management Tensors and operators are Python objects that need to be created, tracked, and destroyed.
- Framework dispatch logic Each operation requires the framework to decide which kernel to run, on which device, and how to manage memory.

Individually, these costs are small — but in deep neural networks with thousands of operations, they add up.

In deployment, models are usually not run inside Python at all.

Instead, they are executed by:

- C++-based runtimes
- Hardware-specific inference engines
- Precompiled binaries on accelerators (GPU, NPU, TPU, FPGA)

These runtimes:

- Execute pre-compiled computation graphs
- Call highly optimized kernels directly
- Avoid Python entirely during inference

As a result, they remove most of the overhead associated with Python and dynamic execution.

Simply moving a model out of Python does not automatically guarantee speedups.

To fully benefit, the model must be:

- Compiled into a static or semi-static graph
- Optimized (operator fusion, memory planning, quantization, layout changes)
- Adapted to the target hardware's instruction set and memory hierarchy

Without this careful compilation step, the deployed runtime may still execute inefficient kernels or suboptimal data flows.

Libraries like cuDNN and oneDNN are heavily optimized for throughput-oriented workloads, which historically means large batch sizes. Large batches increase data reuse (e.g., weights reused across many samples), which improves cache efficiency and amortizes memory access costs. With many parallel threads and large workloads, GPUs/CPUs can overlap memory accesses with computation, effectively hiding memory latency. Many optimized kernels (e.g., GEMM-based convolutions) assume enough work to justify complex tiling, blocking, and scheduling strategies. These libraries were primarily developed to support large-scale training in data centers, where batch sizes of hundreds or thousands are common. So the default optimization targets of these libraries align better with large batches than with batch-1 inference, which is preferable for Real-time setups

Frameworks like PyTorch (in eager mode) execute operation by operation, where each op is dispatched independently; intermediate tensors are materialized in memory; the framework has limited visibility beyond the current operator. Compilers, in contrast, operate on a whole computation graph. All operators and data dependencies are visible; control flow and tensor lifetimes can be analyzed globally. Compilers merge multiple operations into a single kernel to reduce memory traffic and overhead.

With a full graph, compilers can:

- Remove redundant operations
- Fold constants
- Reorder associative/commutative ops
- Eliminate dead branches or unused tensors

Furthermore, different hardware prefers different layouts, GPUs often prefer NCHW, Some accelerators prefer NHWC or blocked layouts, so compilers can choose a single optimal layout, insert minimal layout conversions, avoid unnecessary transposes.

2.5 DNNs Complier Workflow

An ML model compiler takes a *high-level model* (PyTorch / TensorFlow / ONNX) and turns it into efficient executable code for a *specific target* (CPU, GPU, NPU, FPGA, ASIC like Hailo-8).

Stages:

- Stage 1: Frontend (Model ingestion)
  - Input: PyTorch, TensorFlow, JAX
  - Exported formats: ONNX, TorchScript, StableHLO
  - Goal:
    - Convert framework-specific graphs into a compiler IR
    - Normalize control flow, ops, shapes
  - Key tasks:
    - Canonicalize ops (e.g. many conv variants → one canonical conv)
- Stage 2: High-level graph optimizations (framework-agnostic)
- Stage 3: Lowering to tensor programs (IR → loops)
- Stage 4: Hardware-specific optimization & scheduling
- Stage 5: Code generation (backend)
- Stage 6: Runtime & execution

A typical ML execution flow involves translating a model defined in a high-level framework (like TensorFlow, PyTorch, or JAX) into a format suitable for high-performance execution on target hardware. This translation process is orchestrated by the ML compiler and runtime stack (see Fig. **??**)

The Compiler's primary responsibility is to convert high-level representation of a DNN model to an optimized lower-level executable by the target hardware.

Main stages:

- Conversion to an intermediate representation (IR);

- Graph level optimization: operation fusion and simplification, memory planning;
- Tensor/Loop-Level IR: tiling, vectorization, memory optimization;
- Code generation: intruction selection, register allocation, target-specific code emission;

ML runtime manages the dynamic aspects of model execution such as execution orchestration (loading compiled model artifact and managing its execution), memory management (allocation of memory for intermediate tensors), interfacing with hardware devices, task dispatching, and library integration.

Popular compilers and runtime stacks:

- TensorFlow/XLA (Accelerated Linear Algebra): compiles Tensorflow graphs via HLO IR;
- PyTorch 2.x (TorchDynamo, AOTAutograd, Inductor): captures Python bytecode into FX graph representation;
- Apache TVM: stack for optimizing models for CPU, GPU, microcontrollers, FPGAs and ASICS.
- MLIR-based Stacks (e.g., IREE, TensorFlow MLIR)
- ONNX Runtime

2.6 Performance Bottlenecks in ML Inference

A workload is compute-bound if increasing the raw processing power (e.g., higher clock speed, more compute units) directly leads to a proportional decrease in execution time, assuming memory and latency are not limiting factors.

Inference for complex neural networks requires millions of multiply-accumulate (MAC) operations. If hardware is not fully utilized and code not map vectors/tensors effectively, the processing bottlenecks are created.

Many ML operations are memory-bound. This means the execution time is dominated by the time spent transferring data between memory levels (e.g., DRAM to caches, cache levels, host-to-device memory) rather than the computation itself.

Latency bounded operations: single-batch inference, overhear for dispatching operations, pipeline stalls.

These strategies reduce dispatch overhead for frequently executed execution paths:

- **Ahead-of-Time (AOT):** Compile the entire model before execution, as in traditional C/C++ workflows and many edge deployment scenarios.
- **Just-In-Time (JIT):** Compile code or computation graphs at runtime, typically upon first execution, leveraging runtime information such as tensor shapes, data types, and target hardware.

2.7 Hailo Compiler

2.8 Methods for Pruning Deep Neural Network

In a survey [3] authors categorized over 150 studies of pruning neural networks. They chose 8 categories:

- **Sensitivity analysis methods (loss / gradient–aware pruning)**. These methods are valuable as they prune what matters the most for the task and normally provide the highest accuracy-compression trade-off (50-90% accuracy with 1-2% accuracy drop).
  Important works in this category:
  - Taylor Pruning [4];
  - SNIP (Single-shot Network Pruning) [5];
  - GraSP (Gradient Signal Preservation) [6];
  - Movement Pruning [7]

  Although sensitivity-based pruning methods often achieve high unstructured sparsity (50–90%), such sparsity is difficult to exploit on general-purpose hardware due to irregular memory access and limited compiler support, resulting in limited real-world speedups. Consequently, structured pruning methods remain more practical for deployment across diverse hardware platforms.
- **Knowledge distillation methods (teacher–student compression)**. Knowledge Distillation (KD) is a model compression paradigm in which a large, high-capacity model (Teacher) transfers knowledge to a smaller, more efficient model (Student). Instead of training the student solely on hard labels, KD trains it to match the teacher's output behavior, typically using soft targets (probability distributions or logits). KD differs fundamentally from standard understanding of pruning: pruning removes parameters or structures, but KD transfers behavior. By doing this KD is normally does both: improves the accuracy by few percent

compared to the baseline and reduces the number of parameters up to 120 times. Important works:

  - Knowledge Distillation [8];
  - DCP [9];
  - AutoCompress [10];

  Knowledge distillation is a powerful model compression paradigm that transfers predictive behavior from a large teacher model to a compact student model using soft supervision. While classical distillation operates at the output level, recent works extend this idea to intermediate representations and pruning-aware settings. Methods such as discrimination-aware channel pruning integrate auxiliary losses to preserve discriminative power during structured pruning, achieving strong accuracy retention. However, distillation-based approaches are inherently task-dependent, require access to labeled data and a strong teacher, and introduce additional training complexity, limiting their applicability in representation-centric or hardware-agnostic compression scenarios.

- **Similarity and clustering methods** Similarity and clustering–based pruning methods aim to remove redundant filters or channels rather than filters that are individually "unimportant". The core assumption is that modern CNNs are overparameterized and contain filters whose functionality can be approximated or replaced by others with minimal impact on accuracy. Instead of ranking filters solely by magnitude or sensitivity, these methods explicitly model correlation, similarity, or representational overlap among filters. Representative methods include:
  - FilterSketch by Lin et al. [11],
  - Geometric Median Pruning by He et al. [12],
  - ThiNet by Lu et al. [13]

- **Low rank methods** are mostly theoretically grounded methods, effective in convolution-heavy models; Best methods: Show moderate speedups and faster accuracy degradation at high compression

- **Magnitude based pruning methods** are considered simple and cheap to implement models, however, they have limited compression and higher accuracy drops, compared to more complex structures; Best examples are L1 filter pruning (Li et al., 2017) [14], APoZ (Activation sparsity) [15].

- **Architectural design methods (NAS, RL search)**. examples: AMC, AutoSlim, Once-for-All (OFA)

- **Hybrid methods**
- **Quantization methods**: are not exactly pruning methods. They are usually combined with pruning. Mostly based on a transformation of weights from floating point to integer. Important works: Quantization-Aware Training (QAT) [16], Binary / Ternary Networks (XNOR, TWN) [17]

2.8.1 Magnitude based pruning

Magnitude based Pruning Methods are methods based on removing weights, reducing amount of inbound and outbound filters and nodes based on a measure of magnitude of the effect these filters have on the network. Pruning iteration is repeated until the accuracy is starting to drop. First works in this field were adopting the idea of removing the weight lower than a particular threshold.

Han et al. [18] carried out several experiments on LeNet-300-100 and Lenet-5, both trained on MNIST dataset, and showed that weight could be reduced by this method by a factor of 12 without significant drops in accuracy. Experiments based on AlexNet and VGG16 trained on ImageNet show the factor of 9 and 12 respectively. Han et al. note that L_1 regularization, a technique used in machine learning to prevent overfitting by adding a penalty term to the cost function, is better immediately after pruning, but that L_2 norm is better, when the weights of the pruned model are fine-tuned. Experiments also suggest that earlier layers are the most sensitive to pruning and that iterative pruning is better that one-shot pruning (cutting the proportion of weights in one cycle). Another observation from their work is that re-initialization of the weights is not enough to construct and accurate model. It is always better to fine-tune the weights of the pruned model.

Later work by Guo et al. [19] notes that magnitude pruning can lead to premature removal of weight that can become important given removal of other weights. They propose a method called Dynamic Network Surgery to keep track of pruned weights and restore them if they turn to be important. For AlexNet on ImageNet they reach a factor of 17 without compromising accuracy.

The other method proposed by the Frankle and Carbin (2018) is the lottery ticket hypothesis proposed by [20] and further continued by Frankle et al. (2019) in [21]. It formulates a statement, that a trained network contains a subnetwork, which can be trained to be at least as accurate as the original network using no more than the number of epochs used for training the original network. This subnetwork is a so-called winning

lottery ticket. Authors test two pruning methods. The first is a magnitude pruning (p% of the weights), after which the remaining weights are reset to their initial values and are retrained. The second method is using an alternative pruning to iteratively prune $p^{\frac{1}{n}}$ of the weights in n cycles. They experimented on VGG and ResNet and on some bigger models suggested that setting of the weights to those obtained from later iteration of training can be beneficial when high levels of pruning are used (more than 30%).

Another question arises is if these subnetworks are transferable to other tasks. Hubens et al. (2020) [22] show that when a network is trained on a larger data set, such as ImageNet, and transferred and fine-tuned for a different task, pruning can result in a smaller network than if it was trained from scratch on the new task.

Despite the experiments proving that some unstructured pruning methods allow a significant drop of weights, not all hardware can process sparse weight matrices and support speed up inference. Pruning of higher granularity structures (filters and channels) or so-called structured pruning helps to avoid this problem, as many existing toolkits support it. Survey [3] suggests three main directions of structured pruning research:

- **Data dependent channel pruning methods**. A useful channel should respond differently to different inputs. A channel that barely changes for any input is potentially useless.
  Polyak & Wolf (2015) [23] propose two methods - inbound pruning (to reduce the amount of input channels) and "Reduce and Reuse" pruning (to cut output channels).
  The assessment of input channels is performed by applying the network to a sample of images and measuring the variance in a feature map. These measures are ranked and those below a threshold are pruned.
  The "Reduce and Reuse" method targets a reduction in the number of output channels produced by a convolutional layer. The underlying assumption is that output channels exhibiting low variation are less informative and can therefore be removed with minimal impact on performance. A key challenge of pruning output channels is that each removed channel is expected as an input by subsequent layers. To address this issue, the Reduce and Reuse method approximates the removed channels as linear combinations of the remaining ones. This approximation is obtained by solving a least-squares optimization problem that minimizes the reconstruction error between the original and approximated outputs. The resulting transformation is implemented as an additional 1×1 convolutional layer, allowing the network to preserve representational capacity while reducing the number of channels.
  The results from their experiments show that the variance based method is more

effective than use of random pruning.

- **Data independent pruning methods**. Relay on the assumption that properties of the filters and output channels, as the proportion of zeros, influence the structure's importance.
  Hu et al. (2016) [15] propose APoZ-based network trimming, a pruning strategy that does not rely on data-driven measures such as feature map variance or entropy. Instead, their method is based on the observation that a large number of zero activations in an output feature map indicates redundancy. Specifically, they introduce the Average Percentage of Zeros (APoZ) as a metric to quantify the weakness of a filter, with higher APoZ values suggesting lower importance. After each pruning step, the network is fine-tuned, and the authors report that fine-tuning yields better results than retraining the network from scratch. When evaluating the proposed method on VGG-16, the authors achieved a compression rate of 2.59 after six pruning iterations, while 2-3% higher Top-1/Top-5 accuracy than the original VGG-16 model.

- **Optimization based channel approximation pruning methods** propose to remove some channels and then re-optimize the remaining ones instead of deciding which channels are important, so that the layer behaves the same.
  One representative approach is FilterSketch, proposed by Lin et al. (2021) in [11]. Experimental results demonstrate that FilterSketch consistently outperforms first-order magnitude-based pruning methods, FilterSketch removes around 41.5% of the FLOPs and parameters while keeping the top-1 accuracy at 93.19%. Compared to 93.26% by the pre-trained model, the accuracy drop is negligible on CIFAR-10.

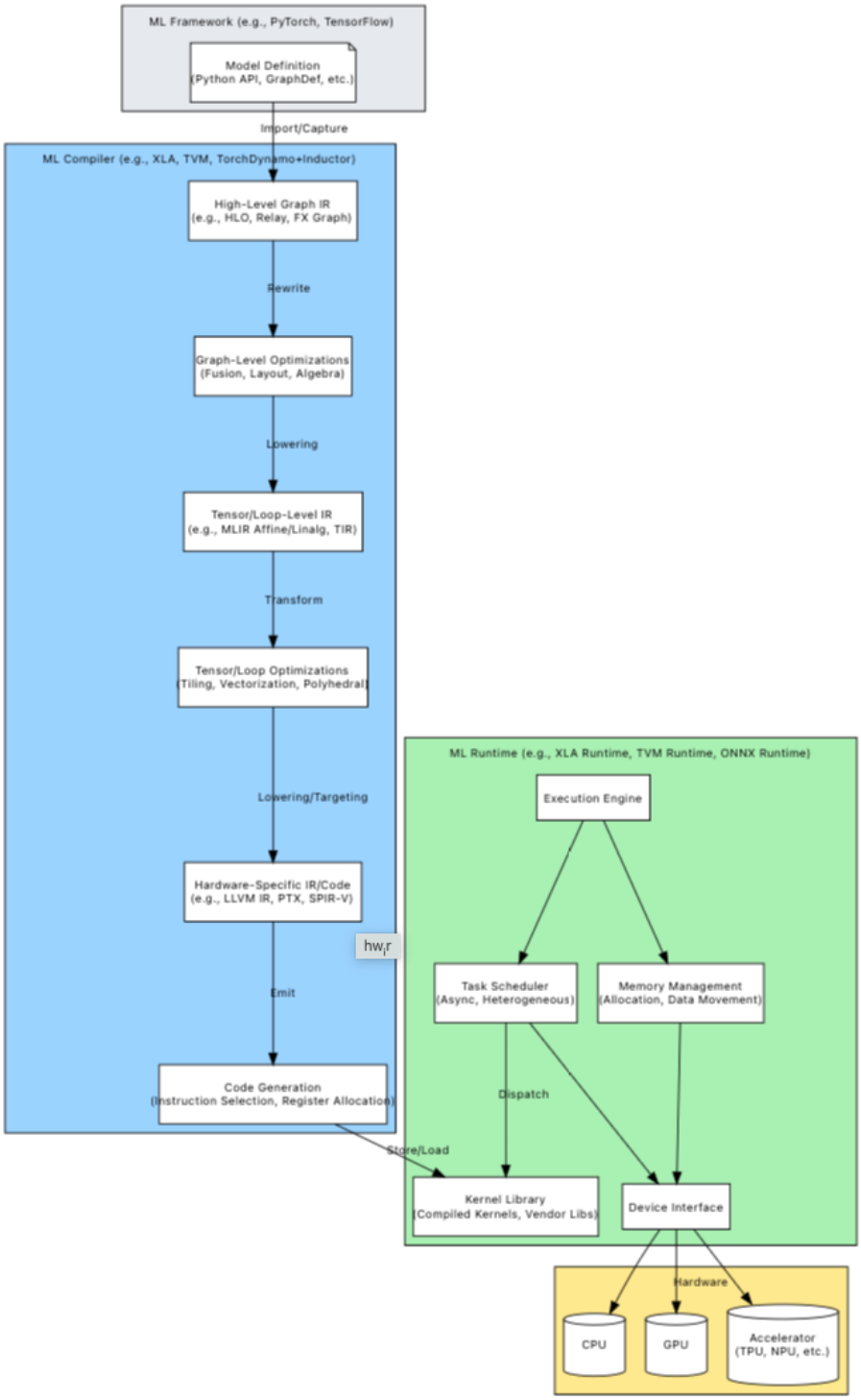

Figure 2.1: Flow of an ML model from definition through compilation and runtime execution on hardware.

3 Theory

3.1 Metrics Description

To evaluate the performance of object detection models, a set of standard metrics is used throughout this report. These metrics assess both localization accuracy and classification quality, providing a comprehensive view of model behavior.

- **Intersection over Union (IoU)**:
  For a predicted bounding box p and a ground-truth box g, the Intersection over Union is defined as:
  $$\text{IoU}(p, g) = \frac{|p \cap g|}{|p \cup g|}.$$
  IoU measures localization accuracy by quantifying the overlap between predicted and ground-truth boxes. It is computed as the ratio of the intersection area to the union area of the two boxes, resulting in a value between 0 and 1. A score of 1 indicates perfect alignment, while 0 indicates no overlap.

  In practice, an *IoU threshold* (e.g., 0.5) is used to determine whether a prediction is correct. Predictions with IoU $\geq \tau$ are considered true positives (TP), while those below the threshold are treated as false positives (FP). IoU serves as a fundamental building block for higher-level evaluation metrics such as precision, recall, and mAP.

- **Precision and Recall**:
  Precision and recall evaluate the quality of classification and detection results. Precision measures the proportion of correct positive predictions:
  $$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}.$$
  Recall measures the proportion of correctly detected instances among all ground-truth objects:
  $$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$
  Here, TP, FP, TN, and FN denote true positive, false positive, true negative, and false negative predictions, respectively. Precision reflects the ability to avoid false detections, while recall captures the ability to detect all relevant objects.

- **Average Precision (AP)**:
  Average Precision summarizes the precision-recall trade-off by computing the area

under the precision-recall (P–R) curve [24]:

$$\mathrm{AP}_\tau = \int_0^1 p(r;\tau)\,dr.$$

The P–R curve is constructed by varying the detection confidence threshold:

- Each detection is assigned a confidence score $s \in [0, 1]$.
- Detections are sorted in descending order of confidence.
- Predictions are progressively included, and precision and recall are recomputed.
- This process generates the precision-recall curve.

AP corresponds to the area under this curve and provides a single scalar measure of detection performance.

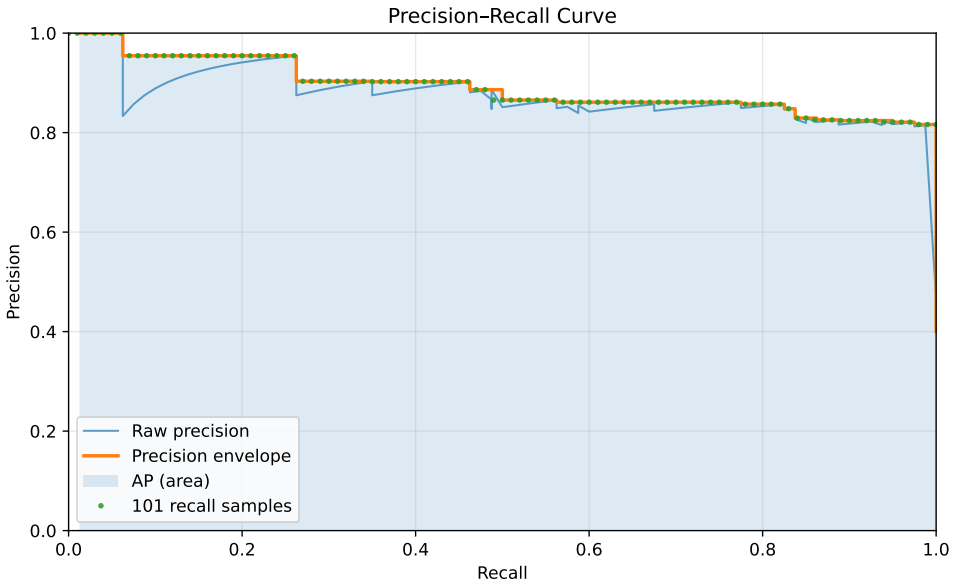

Figure 3.1: Precision–Recall curve.

- **Mean Average Precision (mAP)**:
  Mean Average Precision extends AP to multi-class settings by averaging AP over all object classes:

$$\mathrm{mAP}_{0.50} = \frac{1}{C}\sum_{c=1}^{C} \mathrm{AP}_{c,\,0.50},$$

$$\mathrm{mAP}_{[0.50:0.95]} = \frac{1}{C}\sum_{c=1}^{C}\frac{1}{10}\sum_{k=0}^{9} \mathrm{AP}_{c,\,(0.50+0.05k)}.$$

  Here, C denotes the number of classes. The notation indicates the IoU threshold(s) used to determine true positives. For example, $\mathrm{mAP}_{0.50}$ considers predictions with

IoU ≥ 0.5, while $\text{mAP}_{[0.50:0.95]}$ averages performance across multiple thresholds, providing a more comprehensive evaluation.

3.2 Roofline Model

The Roofline model, introduced by Williams et al. [25], provides an intuitive way to analyze whether a workload is limited by memory bandwidth or computational throughput. It plots performance (e.g., GFLOP/s) versus arithmetic intensity (FLOPs per byte of memory traffic). The model defines two limits: a sloped memory roof determined by peak memory bandwidth, and a flat compute roof determined by peak floating-point throughput. Any kernel's performance is bounded by the minimum of these two ceilings. If a point lies on the sloped region, it is memory-bound; if it lies on the flat region, it is compute-bound. The Roofline model is widely used to analyze whether a workload is limited by memory movement or computation and to guide optimization efforts.

Optimization strategy of a neural network model is dependent on whether our device limits us in the speed of reading/writing to/from memory or the computational speed overall.

- **If the workload is Memory-Bound:**
  This means performance is limited by memory bandwidth, not by compute units. The arithmetic intensity (FLOPs/byte) is low, so the kernel spends most of its time moving data. The goal is then to either increase arithmetic intensity, or reduce memory traffic.
  Optimization strategies:
  - Improve data locality: reuse data in cashes (tiling/blocking); Keep tensors in shared memory (GPU);
  - Fuse operations to avoid writing intermediate tensors to memory (Conv $\rightarrow$ BatchNorm $\rightarrow$ Silu) $\rightarrow$ (Fused Conv-BN-Activation)
  - Reduce Precision (FP32 $\rightarrow$ FP16 $\rightarrow$ INT8 $\rightarrow$ INT4)
  - Reduce tensor size: pruning, structured sparsity, channel reduction, smaller input resolution
  - Improve memory layout: avoid unnecessary copies, channel-last format (NHWC)
- **If the workload is Compute-Bound:**
  This means arithmetic units are saturated. Arithmetic intensity is high enough that bandwidth is no longer limiting. The goal is then to increase compute efficiency. This case is harder, as it requires the work with scheduling, parallelization, kernel

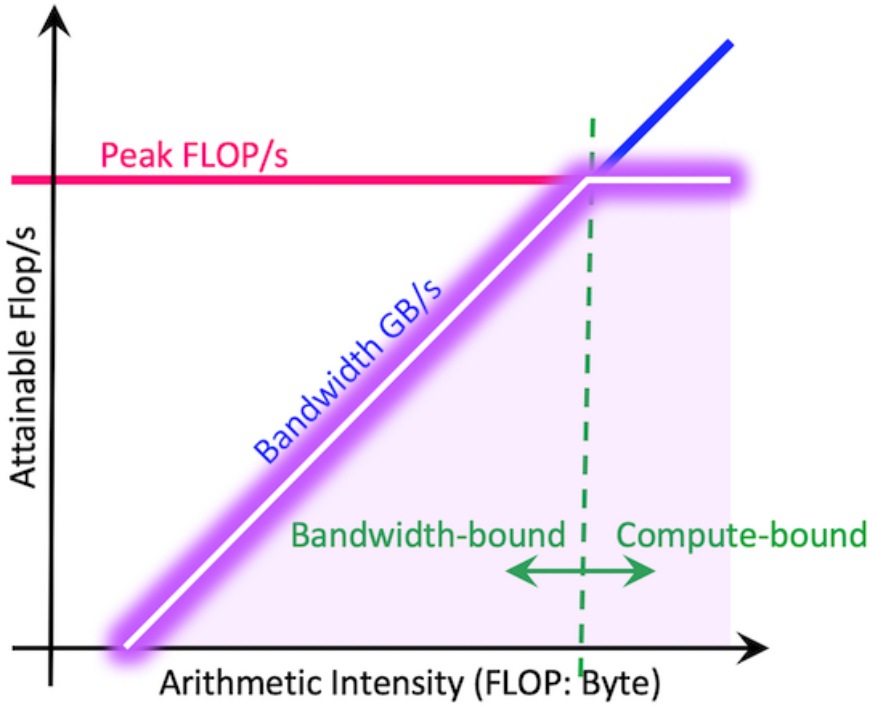

Figure 3.2: A Roofline model illustrating performance limits based on compute capability (horizontal line) and memory bandwidth (sloped line) versus the arithmetic intensity of operations (dots). Operations below the "roof" are limited by either memory bandwidth or compute peak.

optimization and algorithmic improvements.

- **Peak computational performance (in FLOP/s)** is obtained by benchmarking a compute-bound kernel that saturates the floating-point execution units. In practice, this can be achieved using large dense matrix multiplications of size $m \times k$ and $k \times n$, for which the floating-point operation count is given by:

$$\text{FLOPs} = 2 * m * k * n. \tag{3.1}$$

  The sustained peak performance is then computed as

$$P_{\text{peak}} = \frac{\text{FLOPs}}{t_{\text{compute}}}, \tag{3.2}$$

  where t_{compute} denotes the measured execution time.

- **Peak memory bandwidth (in GB/s)** is determined using a streaming memory benchmark. A large device-to-device copy or STREAM-like triad kernel is exe-

cuted over a sufficiently large buffer to avoid cache effects. The sustained memory bandwidth is computed as

$$B_{\text{peak}} = \frac{\text{Bytes transferred}}{t_{\text{memory}}}. \tag{3.3}$$

- **Arithmetic intensity (AI)** is defined as the ratio between the number of floating-point operations and the volume of data transferred to and from main memory:

$$\text{AI} = \frac{\text{FLOPs}}{\text{Bytes}}. \tag{3.4}$$

Since exact DRAM traffic is generally unavailable without hardware performance counters, a lower-bound estimate of memory traffic is commonly used. For convolutional or linear layers, this lower bound consists of the input tensor, weight parameters, and output tensor:

$$\text{Bytes}_{\text{LB}} = \text{Bytes}_{\text{input}} + \text{Bytes}_{\text{weights}} + \text{Bytes}_{\text{output}}. \tag{3.5}$$

This estimate ignores cache reuse and intermediate buffering, thereby providing an upper bound on arithmetic intensity.

- **The achieved performance of a workload** is computed as

$$P_{\text{achieved}} = \frac{\text{FLOPs}}{t_{\text{execution}}}. \tag{3.6}$$

The Roofline bound is then defined as

$$P_{\text{roof}}(\text{AI}) = \min\left(P_{\text{peak}},\ B_{\text{peak}} \times \text{AI}\right). \tag{3.7}$$

If P_{achieved} approaches $B_{\text{peak}} \times \text{AI}$, the workload is memory-bound, if it approaches P_{peak}, the workload is compute-bound. This methodology enables systematic classification of performance bottlenecks and guides optimization strategies such as improving data locality, increasing arithmetic intensity, or enhancing computational parallelism.

3.3 Activation Functions: ReLU and SiLU

Activation functions play a critical role in introducing non-linearity into neural networks, enabling them to learn complex representations. In this work, we consider two commonly used activation functions: Rectified Linear Unit (ReLU) and Sigmoid Linear

Unit (SiLU).

The ReLU activation function was introduced by [26] and is widely used due to its computational simplicity. More recently, smooth activation functions such as SiLU (or Swish) [27] have been proposed, offering improved accuracy at the cost of increased computational complexity.

The ReLU activation is defined as:

$$\mathrm{ReLU}(x) = \max(0, x) \tag{3.8}$$

ReLU is computationally simple, requiring only a comparison and selection operation. This makes it highly efficient on modern hardware, as it can be implemented with minimal arithmetic and memory overhead. Due to its simplicity, ReLU is widely used in performance-critical applications and is well-supported by hardware accelerators.

In contrast, the SiLU activation (also known as the Swish function) is defined as:

$$\mathrm{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \tag{3.9}$$

where $\sigma(x)$ denotes the sigmoid function. Compared to ReLU, SiLU introduces additional non-linearity and has been shown to improve model accuracy in many cases. However, it is computationally more expensive, as it requires evaluation of the exponential function, a division, and a multiplication.

From a computational perspective, the key difference between the two functions lies in their complexity. ReLU involves only a simple thresholding operation, which is typically memory-bound and incurs minimal computational cost. In contrast, SiLU requires multiple floating-point operations, including transcendental functions, making it more compute-intensive and potentially slower on hardware with limited support for such operations.

This difference has practical implications for model optimization. Replacing SiLU with ReLU can reduce inference latency and improve hardware efficiency, especially on edge devices. However, this simplification may lead to a degradation in model accuracy. Therefore, the choice of activation function represents a trade-off between computational efficiency and predictive performance.

While smooth activation functions such as SiLU have been shown to improve accuracy, recent work suggests that simpler functions like ReLU can offer advantages in terms of computational efficiency and activation sparsity [28]. This sparsity can be exploited by hardware and compilers to reduce the number of effective computations, leading to improved inference performance.

4 Methodology

In this work, we explore existing optimization strategies and evaluate their effectiveness on the YOLOv8n model. This model is selected due to its wide applicability to custom object detection tasks, as well as its strong accuracy relative to models of similar size. Additionally, YOLOv8n is widely adopted and well-supported within the community. Its architecture is sufficiently complex to expose challenges faced by modern compilers when optimizing model performance, while remaining lightweight enough to be deployed on a range of edge devices. At the same time, its size enables efficient training and experimentation on commonly available hardware, such as an NVIDIA GeForce RTX 3070 GPU.

The methodology follows a structured analysis pipeline. First, the baseline performance of the model is evaluated on target hardware using the original FP32 PyTorch weights provided by Ultralytics. This step establishes a reference point and helps determine whether there is room for optimization. To guide this analysis, the roofline model is used to assess whether performance is limited by memory bandwidth or computational capacity. If the model operates near the hardware limits, further improvements may require architectural changes or deployment on more suitable hardware. However, in practice, performance is often limited by suboptimal parallelization or inefficient memory usage.

When memory-related bottlenecks are identified, optimization strategies focus on reducing memory traffic and improving data locality. This includes techniques such as precision reduction (e.g., quantization to INT8), improving cache utilization, minimizing host-to-device data transfers, and applying operator fusion to avoid unnecessary intermediate memory operations. For example, combining convolution, normalization, and activation into a single fused operation can significantly reduce memory overhead.

When compute underutilization is observed, optimization efforts shift toward improving parallel execution and kernel efficiency. This includes refining kernel implementations, reducing the number of launched kernels, and optimizing core operations such as matrix multiplication. Profiling tools are used to identify performance-critical layers and understand the underlying causes of inefficiency.

Building on this analysis, the model is evaluated across multiple inference frameworks to compare performance characteristics and identify framework-specific bottlenecks. Quantization techniques are applied where supported, and their impact on both performance and accuracy is assessed. In addition, the sensitivity of the model to structural modifications is investigated through layer pruning and simplification of computationally

expensive operations, such as replacing SiLU activations with simpler alternatives like ReLU.

Finally, advanced optimization approaches are explored, including the use of TensorRT with custom plugins to enable fine-grained control over kernel execution. This allows further investigation of low-level performance characteristics and potential gains beyond standard framework optimizations.

5 Testing and Results

5.1 Experimental Setup

Experiments are conducted on two target edge platforms: NVIDIA Orin Nano and Raspberry Pi 5. Model training and additional experimentation (e.g., pruning and operator substitution) are performed on a host workstation equipped with an NVIDIA GeForce RTX 3070 GPU.

NVIDIA Orin Nano Platform is an embedded AI computing module designed for edge inference and robotics applications. Its SoC combines: ARM-based CPU with 6 Cores, NVIDIA Ampere-based GPU and various I/O. The shared by CPU and GPU LPDDR memory reduces latency and speeds-up edge-workloads. The GPU in Orin Nano supports CUDA, Tensor Cores, Mixed precision (FP32, FP16, INT8) and 2:4 structured sparsity.

Only key hardware characteristics relevant to performance analysis are reported, namely compute capability, memory size, and memory bandwidth.

Table 5.1: Host GPU specifications (training platform)

Property	**Value**
Device	NVIDIA GeForce RTX 3070
Architecture	Ampere
FP32 performance	∼20 TFLOPs
Memory	8 GB GDDR6
Memory bandwidth	∼448 GB/s

Table 5.2: Edge device specifications

Property	**Orin Nano**	**Raspberry Pi 5**
CPU	6-core Arm Cortex-A78AE	4-core Arm Cortex-A76
GPU	Ampere (1024 CUDA cores)	VideoCore VII
FP32 performance	∼0.5–1 TFLOPs (GPU)	
Memory	8 GB LPDDR5	8 GB LPDDR4X
Memory bandwidth	∼68 GB/s	∼25 GB/s
Accelerators	Tensor Cores	None*

* The Raspberry Pi 5 supports external AI accelerators such as the Hailo-8, which can be integrated via PCIe, enabling efficient neural network inference on the device.

5.2 Roofline Model

Constructing an accurate Roofline model for real-world workloads presents several challenges. First, the performance characteristics reported by hardware manufacturers often differ from measured values under practical conditions. Second, the effective behavior of a model depends on the execution framework, including factors such as cache utilization, memory reuse, and implicit optimizations (e.g., operator fusion), which are typically not visible at the model level.

Official YOLOv8 model characteristics are summarized in Table 5.3. The smallest variant, YOLOv8n, contains 3.2M parameters and requires 8.7B floating-point operations for a single forward pass at 640×640 resolution.

Table 5.3: Comparison of YOLO models (trained on COCO) at 640×640 resolution [29].

Model	CPU ONNX (ms)	A100 TensorRT (ms)	Params (M)	FLOPs (B)
v8n	80.4	0.99	3.2	8.7
v8s	128.4	1.20	11.2	28.6
v8m	234.7	1.83	25.9	78.9
v8l	375.2	2.39	43.7	165.2
v8x	479.1	3.53	68.2	257.8

While vendor specifications indicate that the Raspberry Pi 5 provides up to 17 GB/s of memory bandwidth [30], and the Jetson Orin Nano offers up to 102 GB/s [31], measured performance is often significantly lower in practice.

5.2.1 Roofline Analysis on Raspberry Pi 5

Using the methodology described in Chapter 3, the sustained memory bandwidth of the Raspberry Pi 5 was measured at 9.14 GB/s, and peak compute throughput at 69.86 GFLOP/s.

These values define the device Roofline:

$$P(I) = \min\left(69.86,\ 9.14 \cdot I\right),$$

with a ridge point at approximately 7.6 FLOPs/byte, separating memory-bound and compute-bound regimes.

Table 5.4 summarizes representative per-layer performance characteristics of YOLOv8n executed on the Raspberry Pi 5 CPU with the use of eager mode of Pytorch Framework.

Layers with low arithmetic intensity (AI $\approx$ 7–8 FLOPs/byte) achieve limited performance (3–6 GFLOP/s), indicating memory-bound behavior. In contrast, deeper convolutional layers with high arithmetic intensity (AI $>$ 70 FLOPs/byte) achieve 40–60 GFLOP/s, reaching 70–85% of the measured peak compute throughput and exhibiting compute-bound execution.

The distribution focal loss (DFL) convolution in the detection head has extremely low arithmetic intensity (0.47 FLOPs/byte) and achieves only 0.4 GFLOP/s, confirming strong memory limitation.

Table 5.4: Per-layer performance statistics of YOLOv8n on Raspberry Pi 5 (CPU, FP32).

Layer	Time (ms)	AI (FLOPs/byte)	Perf (GFLOP/s)	Regime
0.conv	23.79	7.71	3.7	Memory-bound
2.cv1.conv	8.88	7.99	5.9	Memory-bound
4.m.0.cv2.conv	2.82	70.42	41.8	Compute-bound
6.m.1.cv2.conv	2.06	122.03	57.3	Compute-bound
18.m.0.cv2.conv	1.95	122.03	60.5	Compute-bound
22.cv3.0.1.conv	13.86	170.41	53.2	Compute-bound
22.dfl.conv	2.42	0.47	0.4	Strongly memory-bound
Full Model	358.35	157.76	12.36	Mixed / Overhead-dominated

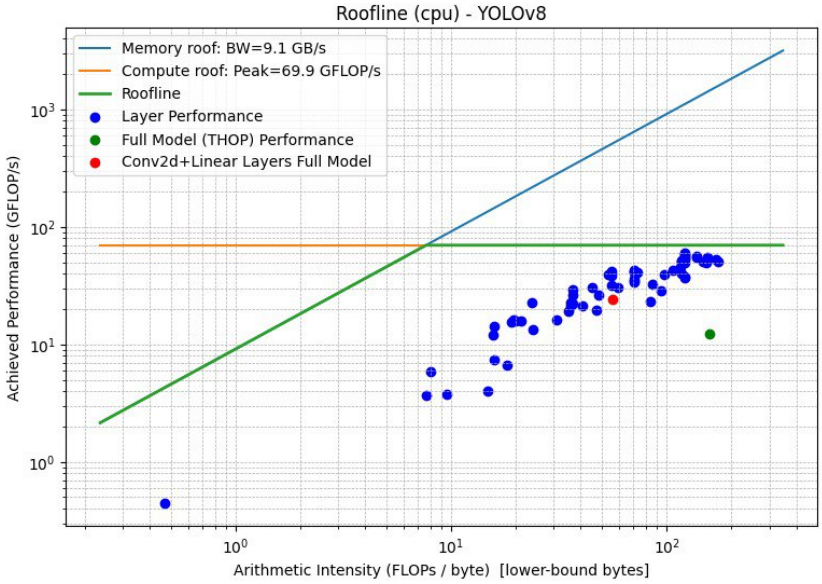

Figure 5.1: Roofline model of YOLOv8n on Raspberry Pi 5 (CPU, FP32).

Although several convolutional layers approach the peak compute throughput, the overall model achieves only 12.36 GFLOP/s, corresponding to approximately 17% of peak performance.

This discrepancy indicates that inference performance is not limited solely by memory bandwidth, but also by system-level and computational inefficiencies. These include kernel launch overhead, framework dispatch costs, synchronization, non-convolutional operations, and inter-layer memory traffic. Thus, the Roofline represents an upper theoretical bound rather than an achievable performance target.

From an optimization perspective, these results suggest that further improvements should focus on enhancing computational efficiency through techniques such as operator fusion, graph-level optimization, kernel tuning, and quantization, rather than exclusively reducing memory traffic.

The observed performance behavior can be further explained by the relationship between layer activation sizes and the cache hierarchy:

- L1: 64 KB per core (instruction and data)
- L2: 512 KB per core
- L3: 2 MB shared
- DRAM bandwidth: $\approx$ 9 GB/s (measured)

In particular, the shared L3 cache of the Raspberry Pi 5 is limited to 2 MB, which constrains the amount of data that can be reused without accessing main memory.

Early layers of YOLOv8n operate on high-resolution feature maps (e.g., 320×320), resulting in activation tensors that significantly exceed the available cache capacity. As a consequence, these layers frequently access DRAM, leading to low arithmetic intensity and memory-bound execution.

In contrast, deeper layers operate on progressively smaller feature maps (e.g., 40×40 or 20×20), allowing their activations to fit within the L3 cache. This improves data locality and reuse, enabling higher arithmetic intensity and more efficient utilization of compute resources, which is consistent with the observed transition to compute-bound behavior.

5.2.2 Roofline Analysis on Jetson Orin Nano

The same analysis was performed on the NVIDIA Jetson Orin Nano GPU. Measured sustained memory bandwidth was 25.42 GB/s, and peak compute throughput reached 810.69 GFLOP/s.

The resulting Roofline model is defined as:

$$P(I) = \min\left(810.69,\ 25.42 \cdot I\right),$$

with a ridge point at:

$$I^* = \frac{810.69}{25.42} \approx 31.9 \text{ FLOPs/byte}.$$

Compared to the Raspberry Pi 5, the Orin Nano exhibits both significantly higher compute throughput and higher memory bandwidth, shifting the ridge point to higher arithmetic intensity. As a result, a larger fraction of YOLOv8n layers operates in the compute-bound regime.

The full model achieves 171.86 GFLOP/s, corresponding to approximately 21% of peak compute throughput, which is higher than the Raspberry Pi 5 but still far from the theoretical limit.

Per-layer analysis shows that many convolutional layers achieve 200–600 GFLOP/s, with several layers approaching or exceeding 800 GFLOP/s. These high-intensity layers (AI $>$ 100 FLOPs/byte) clearly operate in the compute-bound regime.

However, low-intensity layers, such as the DFL convolution (AI $\approx$ 0.47), remain strongly memory-bound, achieving only 3.9 GFLOP/s.

Table 5.5 highlights that, compared to the Raspberry Pi 5, the majority of convolu-

Table 5.5: Representative per-layer performance statistics of YOLOv8n on Jetson Orin Nano (GPU, FP32).

Layer	Time (ms)	AI (FLOPs/byte)	Perf (GFLOP/s)	Regime
model.0.conv	0.85	7.71	104.1	Memory-bound
model.2.cv1.conv	0.26	7.99	200.8	Memory-bound
model.4.m.0.cv2.conv	0.56	70.42	212.3	Compute-bound
model.6.m.1.cv2.conv	0.24	122.03	486.9	Compute-bound
model.12.m.0.cv2.conv	0.24	122.03	497.5	Compute-bound
model.22.cv2.0.1.conv	0.56	137.80	843.7	Compute-bound
model.22.cv3.0.1.conv	0.83	170.41	887.9	Compute-bound
model.22.dfl.conv	0.28	0.47	3.9	Strongly memory-bound
Full Model	25.77	157.76	171.86	Mixed / Overhead-dominated

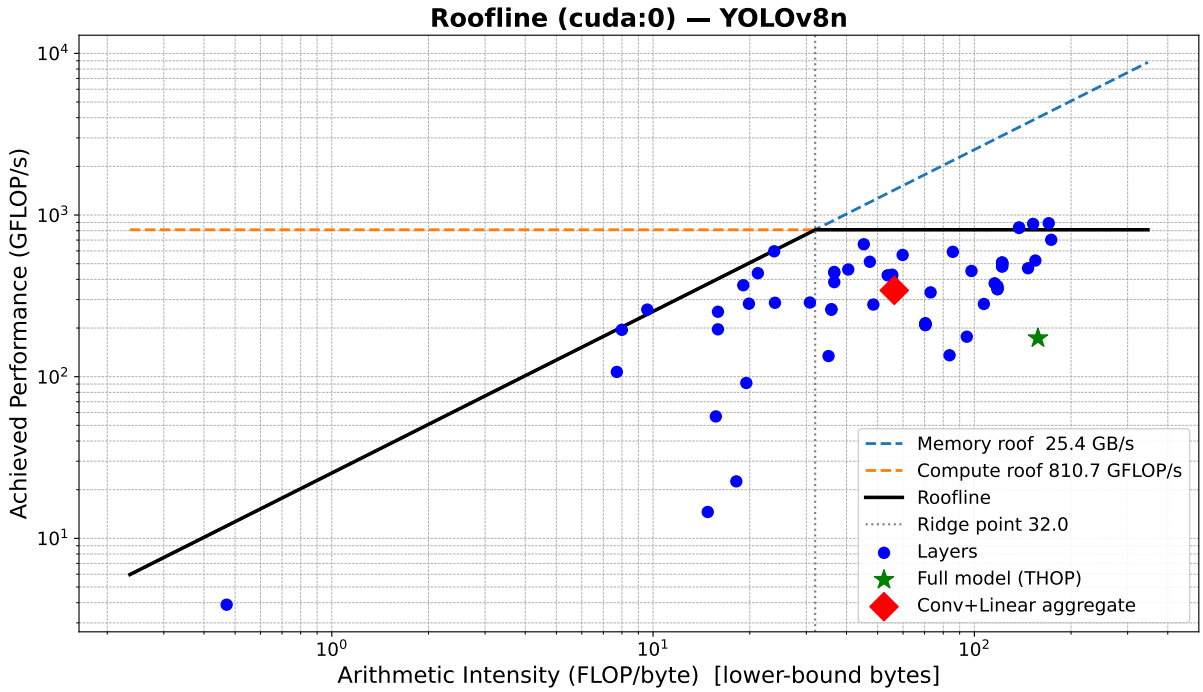

Figure 5.2: Roofline model of YOLOv8n on Jetson Orin Nano (GPU, FP32).

tional layers on the Orin Nano operate in the compute-bound regime, achieving substantially higher performance due to the increased compute capacity and memory bandwidth of the GPU.

However, low–arithmetic-intensity layers, such as the DFL convolution, remain strongly memory-bound even on this more capable platform. Additionally, despite high per-layer performance, the overall model achieves only 171.86 GFLOP/s, indicating the presence of system-level overhead and non-compute-dominated operations.

5.3 Apache TVM

Apache TVM is a modular ML compiler framework that transforms high-level tensor programs into optimized low-level code and runtime artifacts. It is an Open Source Project, mainly developed by a Tianqi Chen's team at Carnegie Mellon University.

Deep learning frameworks make models easy to define, train, and export. However, they do not always produce hardware-efficient binaries for every deployment target. TVM exists to close that gap: it turns model programs into optimized executables across heterogeneous hardware. It also provides deep optimization control over the model.

TVM addresses this with a compiler-first approach: represents computation in intermediate forms, applies transformations, searches optimization spaces, and generates code for many targets.

Core architecture

TVM can be understood as a pipeline:

1. Frontend import: ONNX, PyTorch-exported graphs, TensorFlow, etc.
2. Graph-level IR transformations: operator fusion, layout rewrite, constant folding.
3. Tensor-level lowering and scheduling: loop tiling, vectorization, unrolling, memory placement.
4. Target code generation: LLVM for CPU, CUDA for NVIDIA GPU, other backends.
5. Runtime packaging: graph executor, VM, or AOT runtime.

```
1 def add(a, b):
2     return a + b
```

TVM performs hardware-aware graph partitioning.

5.4 FP32 Runtime Comparison on Jetson Orin

To establish a consistent FP32 baseline across deployment runtimes, a unified benchmark was executed on Jetson Orin. The run was performed on the shared evaluation pipeline, which measures four inference backends under the same input tensor shape and benchmarking procedure:

- TVM (Relax VM, loaded from a precompiled `.so`),
- ONNX Runtime CUDA Execution Provider,
- ONNX Runtime TensorRT Execution Provider,
- PyTorch (Ultralytics model execution).

The configuration (`gpu_fp32.yaml`) used:

- input model: `yolov8n_opset17_fp32.onnx`,
- PyTorch checkpoint: `yolov8n.pt`,
- device: CUDA (`cuda:0`),
- precision: FP32,
- input size: 640×640, batch size $= 1$,
- benchmark protocol: 200 measured runs with 20 warmup iterations.

For TVM, the evaluated module was compiled using an existing MetaSchedule database generated in an earlier tuning stage. The tuning logs indicate a substantial search effort (4282 total trials), suggesting that the measured TVM result reflects a tuned, deployment-oriented build rather than an untuned baseline.

Runtime	Latency [ms]	FPS
ONNX Runtime TensorRT EP	13.5292	73.9142
TVM (Relax VM)	23.5091	42.5368
ONNX Runtime CUDA EP	23.8097	41.9998
PyTorch (Ultralytics)	24.7877	40.3426

Table 5.6: FP32 runtime benchmark results on Jetson Orin (640×640, batch $= 1$).

The results show a clear separation between TensorRT-backed and non-TensorRT execution in this setup. ONNX Runtime with TensorRT EP achieved the lowest latency (13.53 ms), corresponding to 73.91 FPS, while TVM, ONNX Runtime CUDA EP, and

PyTorch formed a second cluster around 23.5–24.8 ms ($\sim$ 40–43 FPS). Relative to ONNX Runtime CUDA EP, TensorRT EP reduced latency by approximately 43.2% (about 1.76$\times$ speedup). TVM and ONNX Runtime CUDA EP were close in performance, with TVM slightly faster in this run.

Overall, this experiment provides a practical FP32 deployment baseline for subsequent quantization analysis. It also indicates that, on Jetson Orin, backend selection can have an impact comparable to model-level optimization: the same FP32 network exhibits markedly different inference speed depending on the runtime execution provider.

5.5 Quantization Influence on YOLOv8n

Statistical significance was evaluated with a paired bootstrap test on validation images (1000 resamples), and confidence intervals were computed using the percentile method. The results indicate that INT8 quantization causes a statistically significant decrease in detection accuracy.

5.5.1 TensorRT FP32 vs INT8 Benchmarking on Jetson Orin

To compare TensorRT performance for FP32 and INT8 inference on Jetson Orin, two engines were exported from the same YOLOv8n model (FP32 and INT8) using Ultralytics export. Benchmarks used batch size 1 and input size 640×640. Reported latency is end-to-end per-batch latency, including host invocation, device execution, synchronization, and output transfer.

Export parameters.

- Model: `yolov8n.pt`
- Input: `640` $\times$ `640`, batch size = 1
- Device: GPU (`device=0`)
- TensorRT workspace: 4 GB
- FP32 engine: `int8=False, half=False`
- INT8 engine: `int8=True, half=False, data=DATA_YAML, fraction=1.0, split="train"`

Early measurements showed strong sensitivity to benchmarking procedure. In particular, when both engines were loaded and benchmarked sequentially in one process, the

second run often benefited from a warmer runtime state. To control this effect, additional experiments were performed with:

- reversed engine order,
- longer warmup (500 iterations),
- fixed Jetson performance settings (`sudo nvpmodel -m 0`, `sudo jetson_clocks`).

Phase	Run mode	FP32 ms	FP32 FPS	INT8 ms	INT8 FPS	INT8 gain
No clocks	dual (int8→fp32)	21.274	47.006	20.473	48.844	3.8%
No clocks	dual (fp32→int8)	21.067	47.469	15.545	64.330	26.2%
No clocks	separate runs	23.122	43.248	20.249	49.385	12.4%
With clocks	dual (int8→fp32)	16.459	60.756	9.876	101.252	40.0%
With clocks	dual (fp32→int8)	16.768	59.639	9.655	103.569	42.4%
With clocks	separate runs	16.588	60.284	10.068	99.328	39.3%

Table 5.7: TensorRT YOLOv8n benchmark summary (batch=1, runs=1000, warmup=500).

Under locked-performance conditions, the INT8 engine achieved approximately 1.65× higher throughput than FP32. Before locking power and clocks, the measured FP32–INT8 gap varied substantially with run order and process isolation. After enabling maximum performance mode and fixed clocks, measurements became more stable and the INT8 advantage increased. The resulting measurements are summarized in Table 5.7.

This pattern suggests that earlier variability was dominated by DVFS and runtime warmup effects. In sequential same-process benchmarking, the later run can benefit from an already initialized CUDA runtime, warmed TensorRT execution context, and elevated GPU/memory clocks. These effects are especially pronounced on Jetson platforms.

Therefore, Jetson benchmarking should report not only numerical performance but also measurement protocol, warmup configuration, and device power state.

`sudo nvpmodel -m 0` selects a high-performance power mode (power budget, online cores, and allowed CPU/GPU/EMC clock limits). `sudo jetson_clocks` then fixes clocks near their maximum allowed values.

Although these settings improve peak performance, they are not always appropriate for continuous use:

- higher power consumption,
- increased heat and possible thermal throttling in long runs,
- more fan noise,

- lower realism for power-managed deployment scenarios,
- reduced responsiveness for mixed workloads,
- greater long-term thermal stress.

A practical approach is to enable them for controlled benchmarking and disable them for routine development or power-constrained deployment.

5.6 GPU Kernel-Level Performance Analysis

To better understand the performance characteristics of the optimized YOLOv8n inference pipeline, kernel-level profiling was performed using NVIDIA Nsight Systems and NVIDIA Nsight Compute on the Jetson Orin Nano platform. The profiling compared TensorRT engines executed in FP32 and INT8 precision modes.

The results provide insight into the underlying execution behaviour of the GPU kernels and reveal several key performance bottlenecks and optimization opportunities.

5.6.1 Kernel Structure of YOLOv8n Inference

Nsight Compute profiling revealed that YOLOv8n inference consists of approximately 35–40 unique GPU kernels executed repeatedly during inference. These kernels can be categorized into several functional groups:

- Tensor Core convolution kernels
- Tensor format conversion kernels
- Tensor permutation kernels
- Pointwise elementwise kernels
- Auxiliary operations such as pooling and softmax

The dominant compute kernels correspond to Tensor Core convolution implementations generated by TensorRT, typically with names such as

```
sm80_xmma_fprop_implicit_gemm_interleaved_i8i8_i8i32_f32
```

which represent implicit GEMM implementations of convolution layers optimized for Ampere Tensor Cores.

INT8 inference uses kernels such as:

```
trt_ampere_int8x4_icudnn_int8x4
```

which exploit INT8 Tensor Core instructions.

5.6.2 Observed Performance Characteristics

The benchmark results show the following performance improvements when using INT8 quantization:

Precision	Latency (ms)	FPS
FP32	16.85	59.35
INT8	9.99	100.12

This corresponds to a speedup of approximately

$$1.68\times$$

when switching from FP32 to INT8 inference.

Despite this improvement, the profiling results show that GPU hardware utilization is still far from optimal.

5.6.3 Occupancy Limitations

Many Tensor Core convolution kernels exhibit very low theoretical occupancy, often in the range of

$$16\% - 33\%$$

This behaviour is caused primarily by high register pressure and large shared memory requirements. For example, several kernels required more than

$$240 \text{ registers per thread}$$

and over

$$80\,\text{kB}$$

of shared memory per block.

High register pressure and large shared memory requirements reduce the number of thread blocks that can be scheduled simultaneously on a Streaming Multiprocessor (SM). Since registers and shared memory are limited hardware resources, each thread block must reserve the required amount before execution can begin.

For example, if a kernel requires a large number of registers per thread or a substantial amount of shared memory per block, only a small number of blocks can reside on the SM at the same time. This limits the number of active warps and therefore reduces the theoretical occupancy of the kernel.

Low occupancy can negatively affect GPU performance because GPUs rely on warp-level parallelism to hide memory and instruction latency. When one warp stalls due to memory access or pipeline dependencies, the scheduler switches execution to another ready warp. However, if only a small number of warps are active due to resource constraints, the scheduler has fewer opportunities to hide these latencies. As a result, the compute units remain idle for longer periods, leading to reduced hardware utilization.

5.6.4 Kernel Launch Granularity

Another recurring issue observed in the profiling results is the small grid size of many kernels. Several kernels were launched with grid sizes smaller than the number of available Streaming Multiprocessors.

For example, kernels with grid sizes of

7 blocks

were observed on a GPU with 8 SMs. That means 12.5% of the GPU is unused for the entire kernel. In such cases the GPU cannot fully utilize all available compute resources.

This effect is commonly referred to as the *tail effect* and is particularly visible when performing inference with batch size 1.

5.6.5 Memory Layout Transformation Overhead

A substantial portion of execution time is spent in kernels responsible for tensor layout transformations and format conversions. Examples include:

```
1 genericReformat::copyVectorizedKernel = layout/type conversion copy
2 CUTENSOR_NAMESPACE::permutationKernel = tensor transpose / dimension reorder /
    permutation
```

```
3 cuInt8::nc32hw32ToNc32hw32 = INT8 packed-layout repacking inside TensorRT's
    channel-blocked formats
```

These kernels do not perform useful neural network computation but instead move or reorder tensor data between different memory layouts required by TensorRT kernels.

Several of these kernels exhibit high memory throughput utilization (often above 70%), indicating that they are primarily memory-bandwidth bound operations.

```
1 genericReformat::copyVectorizedKernel
```

5.6.6 Pointwise Kernel Fragmentation

Another common pattern is the presence of many small pointwise kernels such as:

```
1 generatedNativePointwise
```

These kernels implement operations such as activation functions, elementwise additions, or scaling operations.

While each kernel individually executes quickly, the large number of such kernels introduces kernel launch overhead and reduces overall GPU efficiency.

5.6.7 Overall Performance Bottlenecks

The profiling results suggest that YOLOv8n inference performance is limited by a combination of factors rather than a single dominant bottleneck.

The most significant contributors are:

- Tensor layout conversion overhead
- High register pressure in Tensor Core kernels
- Small kernel grid sizes due to batch size 1
- Fragmentation into many small pointwise kernels
- Moderate compute utilization across many kernels

These observations indicate that the GPU is neither purely compute-bound nor purely memory-bound. Instead, inference performance is limited by kernel launch granularity and resource-heavy kernel implementations.

5.7 Future Optimization Opportunities

Based on the profiling analysis, several optimization directions can be identified for further improving YOLO inference performance.

5.7.1 Operator Fusion

A large number of small pointwise kernels suggests that additional operator fusion could significantly reduce kernel launch overhead. Fusing activation, normalization, and elementwise operations into larger kernels can reduce both kernel count and memory traffic.

Compiler frameworks such as TVM or TensorRT graph optimizations may enable such transformations.

5.7.2 Reducing Tensor Layout Conversions

Memory layout transformation kernels represent a non-negligible portion of total inference time. Reducing the number of tensor format conversions can therefore improve performance.

Possible approaches include:

- ensuring consistent tensor layouts across the network
- minimizing precision conversions
- optimizing tensor format selection during engine compilation

5.7.3 Batch Size Optimization

Small kernel grid sizes observed during profiling indicate that batch size 1 inference does not fully utilize GPU resources.

Increasing the batch size can improve SM utilization and reduce the tail effect. However, this may not be acceptable for latency-sensitive real-time applications.

5.7.4 Model Architecture Optimization

Model architecture also influences GPU efficiency. Potential improvements include:

- reducing small tensor operations
- increasing arithmetic intensity
- restructuring layers to improve GPU utilization

Architectural modifications specifically designed for edge GPUs could further improve throughput.

5.7.5 Compiler-Level Optimization

Given the hardware-aware optimization focus of this work, compiler frameworks such as TVM provide an opportunity for further performance improvements.

Potential directions include:

- auto-tuned convolution schedules
- optimized memory layouts
- kernel fusion
- reduced kernel launch overhead

Compiler-generated kernels may reduce the register pressure and shared memory usage observed in TensorRT kernels.

5.7.6 Mixed Precision Optimization

While INT8 quantization provides significant speed improvements, additional gains may be possible through mixed precision strategies such as:

- FP16 convolution kernels
- selective INT8 quantization of compute-heavy layers
- hybrid precision inference pipelines

These strategies may improve numerical stability while retaining most of the performance benefits of quantization.

5.7.7 Summary

The kernel-level profiling results demonstrate that INT8 quantization significantly improves YOLOv8n inference performance on the Jetson Orin Nano platform. However, substantial optimization potential remains.

Future work should focus on reducing tensor layout conversion overhead, improving kernel fusion, and exploring compiler-driven optimizations to better exploit the available GPU hardware resources.

5.8 YOLO pruning

The main caveat is that once YOLOv8n is converted into a TensorRT engine, TensorRT may fuse layers, insert reformats, and choose different kernels/tactics. So the "slow part" you see in TensorRT is not always a 1:1 match with the original YOLO PyTorch layers. NVIDIA explicitly notes that each TensorRT layer can launch multiple kernels, and extra reformat operations may appear as kernels or device-to-device copies.

6 Discussion

6.1 Discussion Points

6.1.1 Quantization and accelerator deployment

6.1.2 Unexplored hardware configurations and deployment possibilities

7 Conclusion

8 Future Work

9 Description of Use of Artificial Intelligence

In working on this specialization project, I have used artificial intelligence (AI) tools in the following limited ways. I used ChatGPT (OpenAI) to assist with idea development, clarifying concepts, and improving the structure and language of the text (for example, by suggesting alternative formulations, checking grammar, and helping to simplify explanations). AI was also used to get help with technical formatting issues in LaTeX and to debug small code examples by explaining error messages and proposing corrections.

All scientific content, including the problem formulation, choice of methods, implementation, experiments, analysis, and conclusions, has been developed, reviewed, and academically assessed by me. AI tools were not used to generate entire sections of the report, to perform literature searches independently, or to answer the assignment in its entirety. I have not uploaded sensitive or confidential information to the AI tools. I am solely responsible for the academic content of this assignment.

Bibliography

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[3] S. Vadera and S. Ameen, "Methods for pruning deep neural networks," *Ieee Access*, vol. 10, pp. 63280–63300, 2022.

[4] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning convolutional neural networks for resource efficient inference," *arXiv preprint arXiv:1611.06440*, 2016.

[5] N. Lee, T. Ajanthan, and P. H. Torr, "Snip: Single-shot network pruning based on connection sensitivity," *arXiv preprint arXiv:1810.02340*, 2018.

[6] C. Wang, G. Zhang, and R. Grosse, "Picking winning tickets before training by preserving gradient flow," *arXiv preprint arXiv:2002.07376*, 2020.

[7] A. Gordon, E. Eban, O. Nachum, B. Chen, H. Wu, T.-J. Yang, and E. Choi, "Morphnet: Fast & simple resource-constrained structure learning of deep networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 1586–1595, 2018.

[8] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[9] Z. Zhuang, M. Tan, B. Zhuang, J. Liu, Y. Guo, Q. Wu, J. Huang, and J. Zhu, "Discrimination-aware channel pruning for deep neural networks," *Advances in neural information processing systems*, vol. 31, 2018.

[10] N. Liu, X. Ma, Z. Xu, Y. Wang, J. Tang, and J. Ye, "Autocompress: An automatic dnn structured pruning framework for ultra-high compression rates," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, pp. 4876–4883, 2020.

[11] M. Lin, L. Cao, S. Li, Q. Ye, Y. Tian, J. Liu, Q. Tian, and R. Ji, "Filter sketch for network pruning," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 12, pp. 7091–7100, 2021.

[12] Y. He, P. Liu, Z. Wang, Z. Hu, and Y. Yang, "Filter pruning via geometric median for deep convolutional neural networks acceleration," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 4340–4349, 2019.

[13] J.-H. Luo, J. Wu, and W. Lin, "Thinet: A filter level pruning method for deep neural network compression," in *Proceedings of the IEEE international conference on computer vision*, pp. 5058–5066, 2017.

[14] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, "Pruning filters for efficient convnets," *arXiv preprint arXiv:1608.08710*, 2016.

[15] H. Hu, R. Peng, Y.-W. Tai, and C.-K. Tang, "Network trimming: A data-driven neuron pruning approach towards efficient deep architectures," *arXiv preprint arXiv:1607.03250*, 2016.

[16] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2704–2713, 2018.

[17] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "Xnor-net: Imagenet classification using binary convolutional neural networks," in *European conference on computer vision*, pp. 525–542, Springer, 2016.

[18] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural network," *Advances in neural information processing systems*, vol. 28, 2015.

[19] Y. Guo, A. Yao, and Y. Chen, "Dynamic network surgery for efficient dnns," *Advances in neural information processing systems*, vol. 29, 2016.

[20] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," *arXiv preprint arXiv:1803.03635*, 2018.

[21] J. Frankle, G. K. Dziugaite, D. M. Roy, and M. Carbin, "Stabilizing the lottery ticket hypothesis," *arXiv preprint arXiv:1903.01611*, 2019.

[22] N. Hubens, M. Mancas, M. Decombas, M. Preda, T. Zaharia, B. Gosselin, and T. Dutoit, "An experimental study of the impact of pre-training on the pruning of a convolutional neural network," in *Proceedings of the 3rd International Conference on Applications of Intelligent Systems*, pp. 1–6, 2020.

[23] A. Polyak and L. Wolf, "Channel-level acceleration of deep face representations," *IEEe Access*, vol. 3, pp. 2163–2175, 2015.

[24] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[25] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[26] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," *Proceedings of the 27th International Conference on Machine Learning (ICML)*, 2010.

[27] P. Ramachandran, B. Zoph, and Q. V. Le, "Searching for activation functions," *arXiv preprint arXiv:1710.05941*, 2017.

[28] I. Mirzadeh, K. Alizadeh, S. Mehta, C. C. Del Mundo, O. Tuzel, G. Samei, M. Rastegari, and M. Farajtabar, "Relu strikes back: Exploiting activation sparsity in large language models," *arXiv preprint arXiv:2310.04564*, 2023.

[29] Ultralytics, "Yolov8 performance metrics." Ultralytics YOLOv8 documentation. Accessed 2026.

[30] Raspberry Pi Foundation, "Raspberry pi processors." Raspberry Pi Documentation. Accessed: 2026-02-16.

[31] NVIDIA, "Jetson orin nano developer kit datasheet." NVIDIA Datasheet. Accessed: 2026-02-16.

[32] RangeKing, "Brief summary of yolov8 model structure (issue #189)." GitHub, 2023. Ultralytics/ultralytics repository.

A Demonstration (Appendix A)

Table A.1: Per-layer performance statistics of YOLOv8n on Raspberry Pi 5 (CPU, FP32). Regime is derived from the roofline knee $AI_{knee} \approx 7.64$ FLOPs/byte (using measured BW=9.14 GB/s and peak=69.86 GFLOP/s): Memory-bound if AI< 6.88, Knee/transition if 6.88 ≤AI≤ 8.40, Compute-bound if AI> 8.40.

Layer	Time (ms)	AI (FLOPs/byte)	Perf (GFLOP/s)	Regime
model.0.conv	23.79	7.713	3.7	Knee / tion
model.1.conv	17.59	23.955	13.4	Comput bound
model.2.cv1.conv	8.88	7.995	5.9	Knee / tion
model.2.m.0.cv1.conv	5.17	35.899	22.8	Comput bound
model.2.m.0.cv2.conv	5.53	35.899	21.3	Comput bound
model.2.cv2.conv	21.02	9.593	3.7	Comput bound
model.3.conv	11.98	47.291	19.7	Comput bound
model.4.cv1.conv	7.08	15.920	7.4	Comput bound
model.4.m.0.cv1.conv	3.45	70.416	34.1	Comput bound
model.4.m.0.cv2.conv	2.82	70.416	41.8	Comput bound
model.4.m.1.cv1.conv	3.28	70.416	36.0	Comput bound
model.4.m.1.cv2.conv	3.43	70.416	34.4	Comput bound
model.4.cv2.conv	6.62	21.192	15.8	Comput bound
model.5.conv	7.30	85.714	32.3	Comput bound

Continued on n

Layer	Time (ms)	AI (FLOPs/byte)	Perf (GFLOP/s)	Regime
model.6.cv1.conv	3.24	30.769	16.2	Comput bound
model.6.m.0.cv1.conv	3.18	122.034	37.1	Comput bound
model.6.m.0.cv2.conv	3.13	122.034	37.7	Comput bound
model.6.m.1.cv1.conv	2.40	122.034	49.2	Comput bound
model.6.m.1.cv2.conv	2.06	122.034	57.3	Comput bound
model.6.cv2.conv	4.91	40.506	21.4	Comput bound
model.7.conv	5.96	97.959	39.6	Comput bound
model.8.cv1.conv	1.98	48.485	26.5	Comput bound
model.8.m.0.cv1.conv	2.92	118.033	40.4	Comput bound
model.8.m.0.cv2.conv	2.27	118.033	52.0	Comput bound
model.8.cv2.conv	2.46	55.491	31.9	Comput bound
model.9.cv1.conv	1.36	35.165	19.2	Comput bound
model.9.cv2.conv	3.42	59.813	30.7	Comput bound
model.12.cv1.conv	5.18	45.283	30.4	Comput bound
model.12.m.0.cv1.conv	2.22	122.034	53.1	Comput bound
model.12.m.0.cv2.conv	2.01	122.034	58.6	Comput bound
model.12.cv2.conv	3.56	36.641	22.1	Comput bound
model.15.cv1.conv	6.91	23.821	22.8	Comput bound

Continued on n

Layer	Time (ms)	AI (FLOPs/byte)	Perf (GFLOP/s)	Regime
model.15.m.0.cv1.conv	2.90	70.416	40.6	Comput bound
model.15.m.0.cv2.conv	2.76	70.416	42.8	Comput bound
model.15.cv2.conv	5.09	19.085	15.4	Comput bound
model.16.conv	3.02	53.731	39.1	Comput bound
model.18.cv1.conv	2.96	36.641	26.5	Comput bound
model.18.m.0.cv1.conv	2.19	122.034	53.9	Comput bound
model.18.m.0.cv2.conv	1.95	122.034	60.5	Comput bound
model.18.cv2.conv	2.70	36.641	29.1	Comput bound
model.19.conv	2.89	73.096	40.8	Comput bound
model.21.cv1.conv	2.03	55.491	38.8	Comput bound
model.21.m.0.cv1.conv	2.35	118.033	50.2	Comput bound
model.21.m.0.cv2.conv	2.32	118.033	50.7	Comput bound
model.21.cv2.conv	1.87	55.491	42.0	Comput bound
model.22.cv2.0.0.conv	8.48	137.799	55.7	Comput bound
model.22.cv2.0.1.conv	8.35	137.799	56.5	Comput bound
model.22.cv2.0.2	3.71	15.919	14.1	Comput bound
model.22.cv2.1.0.conv	4.30	154.839	54.9	Comput bound
model.22.cv2.1.1.conv	2.14	122.034	55.2	Comput bound

Continued on n

Layer	Time (ms)	AI (FLOPs/byte)	Perf (GFLOP/s)	Regime
model.22.cv2.1.2	1.08	15.681	12.1	Comput bound
model.22.cv2.2.0.conv	2.78	107.063	42.4	Comput bound
model.22.cv2.2.1.conv	1.28	83.721	23.1	Comput bound
model.22.cv2.2.2	0.83	14.798	4.0	Comput bound
model.22.cv3.0.0.conv	11.92	152.381	49.5	Comput bound
model.22.cv3.0.1.conv	13.86	170.414	53.2	Comput bound
model.22.cv3.0.2	5.15	19.874	15.9	Comput bound
model.22.cv3.1.0.conv	5.81	173.494	50.8	Comput bound
model.22.cv3.1.1.conv	3.65	146.939	50.4	Comput bound
model.22.cv3.1.2	1.27	19.506	16.2	Comput bound
model.22.cv3.2.0.conv	3.24	115.663	45.5	Comput bound
model.22.cv3.2.1.conv	1.60	94.737	28.7	Comput bound
model.22.cv3.2.2	0.76	18.161	6.7	Comput bound
model.22.dfl.conv	2.42	0.471	0.4	Strongly memory
Full Model (THOP)	358.35	157.765	12.36	Mixed Overhea dominat

B Demonstration (Appendix B)

YOLOv8n summary: 129 layers, 3,157,200 parameters, 0 gradients, 8.9 GFLOPs.

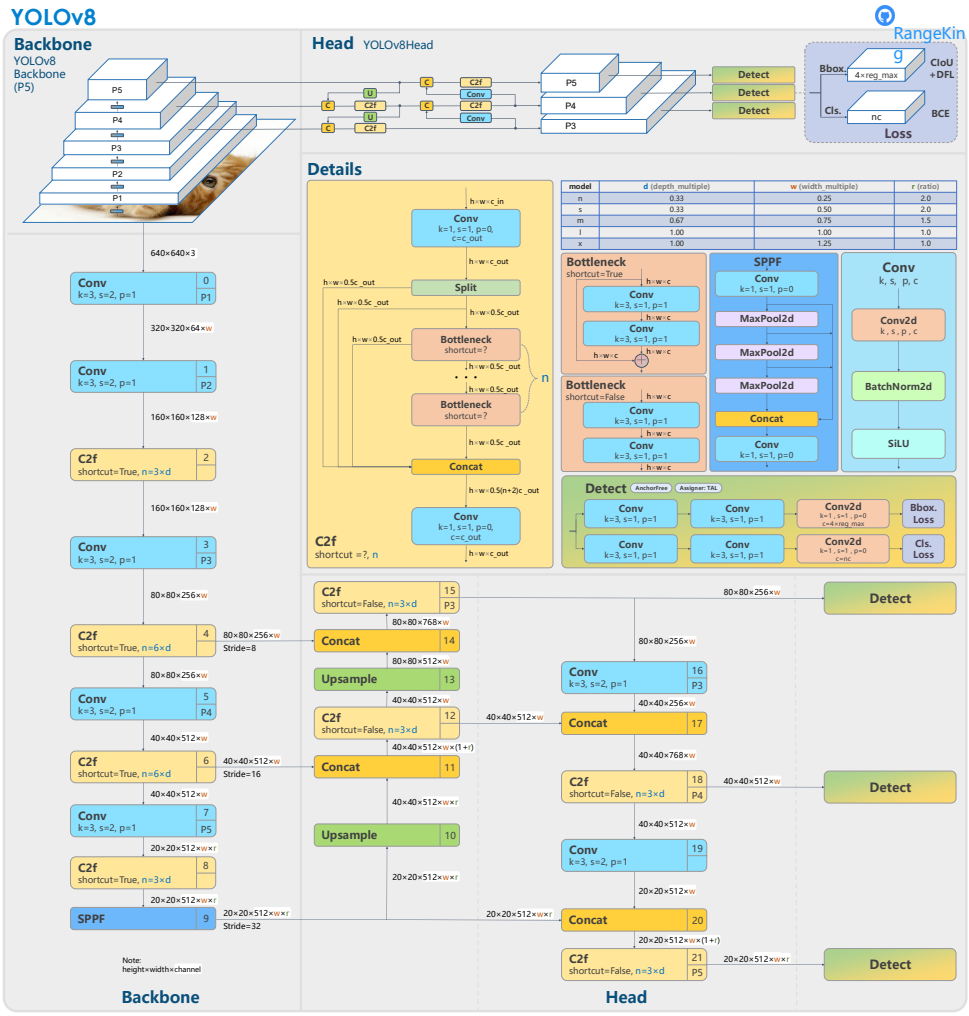

Figure B.1: YOLOv8 structure [32]